

Dashboard ↔ Azure Integration Plan

Connecting your local Next.js dashboard to Azure Container Apps Jobs, then making the dashboard itself remotely accessible from your Mac and phone.

Where things stand

Already built (Azure side):

- Resource group `agent-rg` in `polandcentral`
- Container Registry `jeremylo.azurecr.io`
- Container Apps Environment `agent-env`
- A working Container Apps Job (`hello-job`) with a confirmed end-to-end pipeline:
GitHub push → GitHub Actions build → ACR → scheduled job execution → logs in Log Analytics

Not yet built:

- Any connection between your dashboard and Azure
 - Any authentication on the dashboard
 - Any public/remote hosting of the dashboard
-

Phase 1 — Lock down the dashboard before it goes anywhere public

Goal: Add a password gate so the dashboard can't be used by anyone who finds the URL.

- ☐ Run the Claude Code prompt (already provided) to add middleware-based password auth
- ☐ Generate a strong password + session secret (`openssl rand -base64 24`, twice)
- ☐ Test locally: confirm every page and API route is blocked without logging in
- ☐ Confirm `.env.local` is gitignored and not committed

Status: Ready to execute now.

Phase 2 — Azure read access: view jobs, status, and logs

Goal: Your dashboard can list your Container Apps Jobs and show their recent execution status, without being able to change anything yet (read-only first, so mistakes can't cost you anything).

- ☐ Create the Azure Service Principal (`az ad sp create-for-rbac`) scoped to `agent-rg`
- ☐ Store `AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`, `AZURE_RESOURCE_GROUP` as env vars (local `.env.local` first)
- ☐ Install `@azure/identity` and `@azure/arm-appcontainers` in the dashboard project
- ☐ Add an API route (`/api/jobs`) that lists jobs + their latest execution status
- ☐ Add an API route or extend the above to pull recent log lines via `@azure/monitor-query` (Log Analytics)
- ☐ Build a simple UI table: job name, last run status, last run time
- ☐ Test locally against your real `hello-job`

Claude Code prompt for this phase:

Add a read-only Azure integration to this dashboard. I have Azure service

principal credentials that will be provided as environment variables:

`AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET`, `AZURE_TENANT_ID`,
`AZURE_SUBSCRIPTION_ID`,

`AZURE_RESOURCE_GROUP`.

Install `@azure/identity` and `@azure/arm-appcontainers` if not already present.

Create an API route (server-side only, behind the existing auth middleware)

that:

1. Authenticates using `ClientSecretCredential` from `@azure/identity`
2. Lists all Container Apps Jobs in `AZURE_RESOURCE_GROUP` using
`@azure/arm-appcontainers`

3. For each job, fetches its recent executions and their status

(Succeeded/Failed/Running)

4. Returns this as JSON: [{ name, status, lastRunTime, executions: [...] }]

Then build a simple dashboard page/section that calls this API route and displays the jobs in a table: name, current status, last run time, with a colored badge for status (green/red/gray).

Do not add any write/trigger/deploy functionality yet — this phase is read-only. Add the new env vars to .env.local.example with placeholder values.

Status: Blocked on Phase 1 being complete (route must sit behind auth).

Phase 3 — Trigger jobs manually from the dashboard

Goal: Add a "Run now" button next to each job.

- ☐ Add an API route (`POST /api/jobs/[name]/start`) using `client.jobs.beginStartAndWait`
- ☐ Add a button in the UI wired to that route
- ☐ Add a loading/confirmation state (job triggering isn't instant)
- ☐ Test on `hello-job` — confirm a manual run shows up in Azure

Claude Code prompt for this phase:

Add the ability to manually trigger a Container Apps Job from this dashboard. This builds on the existing read-only Azure integration (same credentials, same `@azure/arm-appcontainers` client).

Add a POST API route at `/api/jobs/[name]/start` that calls

`client.jobs.beginStartAndWait(resourceGroup, jobName)` and returns the

result. Make sure this route is behind the existing auth middleware.

In the UI, add a "Run now" button next to each job in the jobs table.

While the request is in flight, show a loading state on that button.

On success, refresh the jobs list so the new execution shows up. On

failure, show an inline error message.

Status: Depends on Phase 2.

Phase 4 — Deploy new agents from the dashboard

Goal: Create/update a Container Apps Job (i.e. "add a new agent") without touching the Azure CLI directly.

This is the biggest phase — it has two reasonable approaches, pick one:

Option A (simpler): trigger the existing GitHub Actions build, then create the job

- Dashboard calls GitHub's API to kick off your `build-and-push` workflow for a given repo/tag
- Once built, dashboard calls `client.jobs.beginCreateOrUpdate` to create a new Container Apps Job pointing at that image

Option B (more self-contained): dashboard builds and pushes the image itself

- Requires running `docker build` somewhere with access to a Docker daemon — not realistic from a Vercel-hosted serverless function
- Only worth doing if you move the dashboard's backend to something with persistent compute (not recommended given free-tier goal)

Recommendation: Option A. Keep GitHub Actions doing what it's already good at (building), and let the dashboard just orchestrate — trigger builds and create/update jobs via the Azure SDK.

- ☐ Create a GitHub Personal Access Token scoped to trigger workflows (`workflow` scope), store as `GITHUB_TOKEN` env var
- ☐ Add an API route that calls GitHub's "create workflow dispatch" endpoint
- ☐ Add an API route using `client.jobs.beginCreateOrUpdate` to create a new job (name, image tag, cron schedule as inputs)
- ☐ Build a simple form in the UI: repo/image name, cron schedule, job name → submit → calls both routes in sequence
- ☐ Test by "deploying" a second dummy agent end-to-end from the dashboard

Status: Design this in detail once Phases 1–3 are working — it's the most complex piece and easiest to get right after you've built the simpler API routes first.

Phase 5 — Deploy the dashboard to Vercel (remote access begins here)

Goal: Get a live HTTPS URL you can reach from anywhere.

- ☐ Push the dashboard repo to GitHub (if not already there)
- ☐ Go to `vercel.com` → sign in with GitHub → import the repo
- ☐ In Vercel project settings → Environment Variables, add all of:
 - `DASHBOARD_PASSWORD`, session secret (from Phase 1)
 - `AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`, `AZURE_RESOURCE_GROUP` (from Phase 2)
 - `GITHUB_TOKEN` (from Phase 4, if built by then)
- ☐ Deploy — Vercel gives you `your-project.vercel.app`
- ☐ Test the live URL: confirm the password gate works, confirm jobs load, confirm triggering works
- ☐ Every future `git push` to `main` auto-redeploys — same pattern as your agent's CI/CD

Status: Do this once Phase 1 (and ideally 2–3) are solid locally — no reason to deploy an empty shell.

Phase 6 — Mac + phone access

- ☐ **Mac:** nothing to build — bookmark the Vercel URL, done
- ☐ **Phone:** works immediately in any mobile browser at the same URL

- ☐ **Optional polish — PWA:** add a `manifest.json` + icons so it can be added to your phone's home screen and open fullscreen like a native app

Claude Code prompt for the PWA polish (optional, later):

Turn this Next.js app into an installable PWA. Add a `manifest.json` with appropriate name, short_name, icons (generate simple placeholder icons if none exist, at 192x192 and 512x512), theme_color, and display: "standalone".

Link it in the root layout. Confirm this doesn't interfere with the existing auth middleware.

Status: Purely cosmetic — skip until everything else works.

Phase 7 — Optional: custom domain

- ☐ If you own a domain, add it in Vercel project settings → Domains
- ☐ Point DNS records as Vercel instructs
- ☐ Optional entirely — `.vercel.app` works fine indefinitely

Status: Not required, purely aesthetic.

Suggested order of execution

1. Phase 1 (auth) — now
2. Phase 2 (read-only Azure view) — next
3. Phase 3 (manual trigger) — small addition once 2 works
4. Phase 5 (deploy to Vercel) — do this once 1–3 are solid locally, so you get remote access early and can keep iterating live
5. Phase 4 (deploy new agents) — the big one, tackle once the rest is stable
6. Phase 6 (PWA polish) — anytime after Phase 5
7. Phase 7 (custom domain) — anytime, optional

This order gets you a secure, remotely-accessible, functional dashboard (view + trigger) fairly quickly, and saves the most complex piece (deploying new agents) for once the foundation is proven to work.